```
# ------------------------------------------------------------
# effects.py
# ------------------------------------------------------------

"""Visual effects: particles, casings, breach markers, blood decals."""
import math
import random
import pygame

from config import (
    TILE_SIZE, WIDTH, HEIGHT, LETTER_H,
    SCORE_TOP, SCORE_BOT, C_BLOOD, Particle, Casing, audio,
)
from state import STATE

_wall_shadow = pygame.Surface((TILE_SIZE + 8, TILE_SIZE + 8), pygame.SRCALPHA)
pygame.draw.rect(_wall_shadow, (0, 0, 0, 90), (4, 4, TILE_SIZE + 4, TILE_SIZE + 4))

_bullet_glow = pygame.Surface((16, 16), pygame.SRCALPHA)
pygame.draw.circle(_bullet_glow, (255, 200, 100, 80), (8, 8), 8)

_big_door_shadow = pygame.Surface(
    (TILE_SIZE * 3 + 8, TILE_SIZE + 8), pygame.SRCALPHA
)
pygame.draw.rect(
    _big_door_shadow, (0, 0, 0, 90),
    (4, 4, TILE_SIZE * 3 + 4, TILE_SIZE + 4),
)

def add_blood(x, y, life=600, size_range=(8, 14)):
    STATE.blood_decals.append([x, y, life, random.randint(*size_range)])

def spawn_particles(x, y, color, count=8):
    # Graphics > Particle Density scales every spawn site uniformly.
    # Density 0 still emits at least 1 particle so FX never fully disappears.
    try:
        from user_settings import SETTINGS
        density = SETTINGS.particle_density
    except Exception:
        density = 1.0
    actual = max(1, int(round(count * density)))
    for _ in range(actual):
        STATE.particles.append(Particle(x, y, color))

def update_particles():
    for p in STATE.particles[:]:
        p.update()
        if p.life <= 0:
            STATE.particles.remove(p)

def update_casings(solid_walls):
    for c in STATE.casings[:]:
        c.update(solid_walls)
        if c.life <= 0:
            STATE.casings.remove(c)

def update_floating_texts():
    for ft in STATE.floating_texts[:]:
        if len(ft) < 6 or ft[5] != "pin":
            ft[1] -= 0.8
        ft[4] -= 1
        if ft[4] <= 0:
            STATE.floating_texts.remove(ft)

def update_breach_markers():
    for marker in STATE.breach_markers[:]:
        marker[2] -= 1
        if marker[2] <= 0:
            STATE.breach_markers.remove(marker)

def shake(amount):
    """Max-clamp screen shake trigger.

    Always use this - never write ``STATE.screen_shake = N`` directly. Direct
    assignment cancels any in-flight larger shake (e.g. a player hit at amp 8
    mid-breach would stomp the breach bang at amp 10). This helper preserves
    the larger value so smaller triggers never cause a visible stutter.
    """
    if amount > STATE.screen_shake:
        STATE.screen_shake = amount

def hit_stop(seconds):
    """Trigger a hit-stop freeze (GDD Pass 3).

    Per reference/game-design.md "hit-stop": on heavy strikes, pause the
    simulation for ~60 ms so the impact lands with weight. The freeze is
    gated in scorched.py's main loop - when ``STATE.hit_stop_timer > 0``,
    ``gl.game_update(keys)`` is skipped, but rendering still runs (so the
    player sees the moment of impact frozen on screen).

    Always use this helper - never write ``STATE.hit_stop_timer = N``
    directly. The helper max-clamps so a shorter hit-stop cannot truncate
    a larger in-flight one (e.g. a 60 ms melee kill hit-stop should not
    be cut short by a follow-up 40 ms bullet impact). Use the same
    max-clamp discipline as ``shake()``.

    ``seconds`` is the freeze duration in seconds. For 60Hz spec, ~0.05s
    (50 ms / 3 frames at 60 FPS) is the canonical value for heavy strikes;
    use ~0.03s for mid-piece hits (sniper, blast). Callers should pass
    constants values from settings/constants, not magic numbers.
    """
    if seconds > STATE.hit_stop_timer:
        STATE.hit_stop_timer = seconds

def burst(x, y, color, count=20, speed_mult=1.0, life_min=20, life_max=40, jitter=0):
    """Unified particle burst.

    Both shooters previously had their own spawn loops: shooter.py used raw
    ``Particle(x, y, color)`` (life 18-35, no jitter, no speed scaling) while
    shooter4.py used a local ``_burst()`` helper (life 18-50, +/-8 px jitter,
    speed_mult). This function unifies the two so the 60 parameters (which
    produce the richer-looking burst). The user's ``SETTINGS.particle_density``
    setting is respected via the same scaling as ``spawn_particles``.
    """
    try:
        from user_settings import SETTINGS
        density = SETTINGS.particle_density
    except Exception:
        density = 1.0
    actual = max(1, int(round(count * density)))
    for _ in range(actual):
        jx = random.randint(-jitter, jitter) if jitter else 0
        jy = random.randint(-jitter, jitter) if jitter else 0
        p = Particle(x + jx, y + jy, color)
        p.dx *= speed_mult
        p.dy *= speed_mult
        p.life = random.randint(life_min, life_max)
        STATE.particles.append(p)

def flash(frames=4):
    STATE.gl_flash_timer = frames

def play_impact(name, source_pos, base_volume=0.8, listener=None):
    """Spatial audio for combat impacts.

    Both shooters "should" use 3D positional audio for bullet impacts so the
    player can hear "where" the hit landed. shooter.py did this; shooter4.py
    regressed to flat ``audio.play()``. This helper restores the 3D path
    while gracefully degrading to flat playback when no listener is available
    (e.g. player is dead).
    """
    if listener is None:
        if STATE.player and not getattr(STATE, "player_dead", False):
            listener = (STATE.player.rect.centerx, STATE.player.rect.centery)
        else:
            audio.play(name, volume=base_volume)
            return
    audio.play_3d(name, source_pos, listener, base_volume=base_volume)

# ------------------------------------------------------------
```